

实验二：Flutter 跨平台应用开发

实验项目基本信息

- 实验编号：d20301035102、d20301009202
- 学时分配：4 学时
- 实验类型：验证型
- 每组人数：6 人
- 对应课程目标：课程目标 1

核心步骤列表

实验概览



1. 需求分析阶段：编写登录功能需求文档（输入验证规则、页面跳转逻辑）

- 2. **AI 辅助编码阶段**：使用 TRAE 生成登录页面骨架代码并完善业务逻辑
- 3. **测试验证阶段**：编写输入验证测试用例，验证边界条件
- 4. **部署运维阶段**：打包调试版本，记录跨平台实现对比报告

开发任务清单

任务阶段	主要内容	关键技术点
Flutter 登录	Dart + Material3	TextField、Navigator、State Management
多平台测试	跨平台部署	Android/iOS/Web/Linux/macOS/Windows
组内对比	技术分享	跨平台优势分析

成功标准

- Flutter 登录页面功能完整
- 多平台部署正常
- 页面导航与数据传递正常
- 输入验证逻辑正确
- 跨平台实现对比报告完成

实验任务与案例

核心任务

使用 Dart 实现 Flutter 登录页面（TextField 输入验证、Navigator 页面导航与数据传递），实现跨平台部署，组内成员选择不同平台进行测试，完成后进行技术对比分享。

实战案例

开发"智慧校园"登录模块，学生在登录页面输入学号和密码，系统验证成功后跳转到个人中心页面，显示个性化的欢迎信息和用户资料。

第一课时：Flutter 登录页面设计（2 学时）

2.1 需求分析阶段

步骤 1：编写需求文档

1. 功能需求

- 用户名输入（学号）
- 密码输入
- 登录按钮
- 输入验证（不能为空、长度限制）

2. 界面需求

- Material3 风格
- 输入框占位提示
- 密码隐藏显示

3. 跳转需求

- 登录成功 → 个人中心
- 传递用户名数据

2.2 Flutter 项目创建

步骤 1：创建项目

使用 Flutter CLI 命令创建新项目，选择 Material3 风格，配置项目名称和路径。

2.3 AI 辅助编码

步骤 1：生成登录页面代码

使用 TRAE 描述需求：Flutter 登录页面，TextField 输入验证，Navigator 页面导航，AI 生成代码片段，手动完善业务逻辑。

步骤 2：实现登录页面

设计登录页面布局，包含学号输入框、密码输入框和登录按钮，实现输入验证逻辑。

步骤 3：创建个人中心页面

设计个人中心页面布局，显示欢迎信息和用户资料，接收登录页面传递的用户名数据。

步骤 4：配置路由管理

设置应用初始路由和页面导航规则，实现页面间数据传递。

第二课时：多平台测试与对比（2 学时）

2.5 多平台部署

步骤 1：Android 部署

使用 Flutter CLI 命令编译并部署到 Android 模拟器或真机。

步骤 2：iOS 部署

使用 Flutter CLI 命令编译并部署到 iOS 模拟器或真机。

步骤 3：Web 部署

使用 Flutter CLI 命令编译并部署到 Web 浏览器。

步骤 4：桌面平台部署

使用 Flutter CLI 命令编译并部署到 Windows、macOS 或 Linux 桌面平台。

2.6 测试验证阶段

步骤 1：编写测试用例

- 1. 输入测试
- 2. 为空用户名格式测试
- 3. 密码长度测试
- 4. 正确凭证测试

步骤 2：执行测试

- 1. 在各平台模拟器上运行
- 2. 验证边界条件
- 3. 记录测试结果

2.7 跨平台实现对比

步骤 1：整理对比报告

对比项	Flutter	Android	iOS
开发语言	Dart	Kotlin	Swift
UI 框架	Material3	XML/Compose	SwiftUI
页面跳转	Navigator	Intent	NavigationLink
数据传递	ModalRoute.arguments	Intent.putExtra	@Binding
代码复用	100%	0%	0%
开发效率	高	中	中

步骤 2：组内技术分享

- 1. 各成员介绍实现方案
- 2. 讨论跨平台优势
- 3. 总结技术选型考虑

总结与思考

实验总结

- 掌握 Flutter Dart 登录页面开发
- 理解跨平台应用实现
- 学会页面导航与数据传递
- 完成跨平台实现对比分析

课后思考

1. 跨平台开发相比原生开发的优势和劣势
2. 如何选择跨平台开发 vs 原生开发
3. AI 辅助代码生成对开发效率的影响